

ARC-AGI SIN PREENTRENAMIENTO

Isaac Liao
Carnegie Mellon University

Albert Gu
Carnegie Mellon University

Resumen

La sabiduría convencional en la era de los grandes modelos de lenguaje (LLM) sostiene que resolver rompecabezas visuales similares a pruebas de coeficiente intelectual, como los del benchmark ARC-AGI-1, requiere capacidades derivadas de un preentrenamiento masivo.

Como respuesta a esta idea, presentamos **CompressARC**, un modelo de únicamente **76 000 parámetros** que no utiliza ningún tipo de preentrenamiento y que resuelve el **20 %** de los rompecabezas del conjunto de evaluación minimizando la **Longitud Mínima de Descripción (Minimum Description Length, MDL)** del rompecabezas objetivo exclusivamente durante el tiempo de inferencia.

El principio MDL dota a CompressARC de unas capacidades de generalización extremadamente elevadas, poco habituales en aprendizaje profundo. Hasta donde sabemos, CompressARC es el único método basado en aprendizaje profundo para ARC-AGI en el que el entrenamiento se realiza únicamente sobre una sola muestra: el propio rompecabezas que se desea resolver durante la inferencia, eliminando previamente la información correspondiente a la solución final.

Además, CompressARC no se entrena utilizando el conjunto de entrenamiento proporcionado por ARC-AGI.

Bajo estas condiciones extremadamente limitadas en cuanto a disponibilidad de datos, normalmente no esperaríamos que pudiera resolverse ningún rompecabezas. Sin embargo, CompressARC consigue resolver una distribución diversa de problemas creativos de ARC-AGI, lo que sugiere que el principio de Longitud Mínima de Descripción constituye una vía alternativa y viable para producir inteligencia, además del enfoque convencional basado en preentrenamiento.

1. Introducción

El benchmark **ARC-AGI-1** consiste en una colección de rompecabezas de razonamiento visual abstracto diseñados para evaluar la capacidad de un sistema para adquirir rápidamente nuevas habilidades a partir de una cantidad mínima de datos de entrada (Chollet, 2019).

Los avances recientes en razonamiento basado en grandes modelos de lenguaje han demostrado capacidades impresionantes para adquirir nuevas habilidades. Sin embargo, estos sistemas siguen dependiendo de enormes cantidades de datos de preentrenamiento.

En este trabajo exploramos cuál es la cantidad mínima de información realmente necesaria para abordar ARC-AGI mediante la introducción de **CompressARC**, un método de resolución derivado del principio de **Longitud Mínima de Descripción (MDL)** (Rissanen, 1978).

CompressARC realiza todo su aprendizaje durante el tiempo de inferencia y alcanza una precisión del **20 %** sobre los rompecabezas del conjunto de evaluación de ARC-AGI-1 utilizando únicamente el propio rompecabezas que está resolviendo como fuente de datos.

La clave de la extraordinaria eficiencia de datos de CompressARC reside en formular el problema como un ejercicio de **code golf**: encontrar el programa autónomo más corto posible capaz de generar como salida el conjunto completo de datos de ARC-AGI, rellenando arbitrariamente las cuadrículas cuya solución aún se desconozca.

Siguiendo la navaja de Occam, cabe esperar que el programa más corto complete dichas cuadrículas de la forma más razonable posible, es decir, utilizando soluciones "correctas" que respeten las reglas implícitas del rompecabezas.

Un programa excesivamente simple podría limitarse a almacenar una cadena de texto con todos los datos del rompecabezas (junto con soluciones arbitrarias) e imprimirla como salida. Sin embargo, dicho programa sería demasiado largo, lo que implica que las soluciones producidas serían incorrectas.

Por otro lado, encontrar el programa absolutamente más corto requeriría enumerar exhaustivamente una enorme cantidad de programas candidatos, algo computacionalmente inviable (Solomonoff, 1964; Hutter, 2005).

CompressARC propone un nuevo equilibrio: sobreajusta una red neuronal específicamente al rompecabezas para comprimirlo dentro de las matrices de pesos de la red. Posteriormente, estos pesos pueden codificarse directamente dentro del programa en lugar de almacenar el rompecabezas completo, utilizándose después para reconstruir los rompecabezas memorizados junto con sus soluciones.

Contabilizando cuidadosamente el número de bits necesarios para representar dichos pesos, esta técnica transforma la búsqueda combinatoria del mejor programa en un problema de optimización diferenciable, permitiendo minimizar la longitud del programa en un tiempo razonable y generar predicciones de alta calidad para las soluciones.

Este planteamiento conserva varias propiedades especialmente atractivas del enfoque original basado en code golf, todas ellas novedosas dentro de las soluciones neuronales para ARC-AGI:

- **Sin preentrenamiento.** Dado que el rompecabezas objetivo ya está disponible, no existe una fase previa de entrenamiento.

- **Aprendizaje durante la inferencia.** La longitud del programa se minimiza exclusivamente durante la inferencia optimizando los pesos de la red con respecto al rompecabezas objetivo.
- **Necesidad mínima de datos.** Siguiendo la navaja de Occam, asumimos que el programa más corto posee una fuerte capacidad de generalización y utilizamos únicamente el propio rompecabezas, sin cargar información adicional en memoria.

A pesar de no utilizar nunca el conjunto de entrenamiento, no realizar ningún tipo de preentrenamiento y disponer únicamente de **76 000 parámetros**, CompressARC consigue generalizar hasta resolver el **20 %** de los rompecabezas del conjunto de evaluación y el **34,75 %** del conjunto de entrenamiento, un rendimiento que sería imposible para los métodos tradicionales de aprendizaje profundo bajo estas restricciones.

El buen rendimiento de CompressARC en este escenario sugiere que trasladar los principios de compresión de información a otros ámbitos con escasez de datos —como el descubrimiento de fármacos o el diseño de proteínas— podría permitir obtener capacidades significativamente superiores también en esas aplicaciones.

El resto del artículo está organizado de la siguiente forma:

- La Sección 2 introduce el benchmark ARC-AGI.
- La Sección 3 presenta la formulación del problema.
- La Sección 4 describe la arquitectura de CompressARC.
- La Sección 5 expone los resultados experimentales.
- La Sección 6 analiza las implicaciones del trabajo.

2. Antecedentes: el benchmark ARC-AGI

ARC-AGI-1 es un benchmark de inteligencia artificial diseñado para evaluar la capacidad de un sistema para adquirir nuevas habilidades a partir de un número muy reducido de ejemplos (Chollet, 2019).

Cada rompecabezas del benchmark está gobernado por una regla oculta distinta que el sistema debe descubrir y aplicar a una cuadrícula de entrada formada por colores para producir la cuadrícula objetivo correspondiente.

Estas reglas ocultas utilizan conceptos como:

- reconocimiento de objetos;
- comportamiento orientado a objetivos;
- números y conteo;
- geometría básica;
- topología.

Para ayudar al sistema a descubrir la regla, cada rompecabezas proporciona varios pares de ejemplos entrada–salida. No se facilita ninguna información adicional.

La Figura 1 muestra tres ejemplos de rompecabezas del conjunto de entrenamiento de ARC-AGI-1.

Figura 1. Tres ejemplos de rompecabezas de ARC-AGI-1.

(a) Regla oculta: desplazar cada objeto un píxel hacia la derecha, excepto los bordes inferior y derecho del propio objeto.

(b) Regla oculta: reducir el tamaño del objeto grande y asignarle el mismo color que los puntos dispersos.

(c) Regla oculta: prolongar la línea verde hasta alcanzar la línea roja, girando cuando encuentra un obstáculo.

Mientras que las soluciones basadas en grandes modelos de lenguaje entrenados con datos a escala de Internet han alcanzado un **87,5 %** de precisión en este benchmark (Chollet, 2024), y las redes neuronales entrenadas únicamente con datos de ARC-AGI han logrado un **40,3 %** (Wang et al., 2025), CompressARC lleva el enfoque de minimización de datos hasta el extremo, entrenándose exclusivamente sobre el propio rompecabezas de prueba.

El Apéndice L proporciona información adicional sobre el benchmark ARC-AGI-1.

Asimismo, el Apéndice A contiene una revisión ampliada del trabajo relacionado, incluyendo otros enfoques desarrollados para resolver ARC-AGI.

A lo largo del resto del artículo nos referiremos generalmente a **ARC-AGI-1** simplemente como **ARC-AGI**.

3. Método

CompressARC intenta resolver el problema de comprimir los datos en un programa lo más corto posible, de manera que permita obtener las soluciones de los rompecabezas manteniendo, al mismo tiempo, una búsqueda del programa computacionalmente viable.

En este planteamiento, el programa debe ser completamente autónomo, no recibir ninguna entrada y producir como salida el conjunto completo de rompecabezas de ARC-AGI, incluyendo cualquier solución que haya sido rellenada.

Normalmente resulta inviable realizar una búsqueda algorítmica exhaustiva para encontrar el programa absolutamente más corto, ya que sería necesario explorar un número gigantesco de programas de longitud creciente hasta hallar uno cuya salida coincidiera exactamente con nuestros requisitos.

CompressARC hace que esta búsqueda sea mucho más manejable restringiéndola a un subespacio de programas cuidadosamente diseñado.

Nuestra estrategia general, que iremos desarrollando en las siguientes secciones, consiste en lo siguiente:

- Definimos un espacio de programas mediante un **programa plantilla** (Algoritmo 1), que contiene numerosos espacios vacíos destinados a almacenar valores codificados directamente ("semillas" o *seeds*). Estas semillas completan el programa para que pueda ejecutarse y generar los rompecabezas con sus soluciones.
- Definimos un algoritmo de búsqueda (Algoritmo 2) que explora múltiples semillas posibles para rellenar dichos espacios y selecciona aquellas que producen el programa más corto posible, imponiendo únicamente la restricción de que los rompecabezas generados coincidan con los rompecabezas conocidos, mientras que las soluciones permanecen libres. Dado que la mayor parte del tamaño del programa corresponde precisamente a las semillas, basta con contabilizar la longitud total de estas para aproximar la longitud del programa completo.
- Solamente una parte del Algoritmo 2 es realmente necesaria para producir las soluciones; el resto únicamente sirve para construir la representación comprimida del rompecabezas. Para resolver los rompecabezas conservamos únicamente esa parte esencial y aproximamos la búsqueda mediante descenso por gradiente (Algoritmo 3). Las aproximaciones diferenciables utilizadas para estimar la longitud de las distintas semillas son análogas a los diferentes componentes de la función de pérdida de un Autoencoder Variacional (VAE).

Formato de los datos

Cada rompecabezas se representa mediante un tensor con forma:

```
[n_example, width, height, 2]
```

que contiene el color de cada píxel de las dos cuadrículas correspondientes a cada par entrada–salida.

Aquí:

- n_example** representa el número total de pares entrada–salida del rompecabezas, incluyendo el par de prueba cuya salida es desconocida.

Las dimensiones indicadas en esta sección tienen únicamente un propósito explicativo; el formato real utilizado por la implementación se presenta posteriormente, en la Sección 4.

Un primer intento ingenuo de minimizar el tamaño del programa consistiría simplemente en escribir todos los rompecabezas dentro de una enorme cadena de texto e imprimirla.

Posteriormente podrían introducirse optimizaciones eliminando redundancias mediante bucles u otras construcciones del lenguaje.

Nuestro método adopta una estrategia distinta, descrita en las siguientes subsecciones.

3.1 Restricción del espacio de programas

Derivamos CompressARC seleccionando un subespacio concreto de programas definido por una plantilla (Algoritmo 1), que posteriormente se completa sustituyendo determinados valores codificados directamente dentro del programa.

Cada rompecabezas se genera de forma independiente, ejecutando las siguientes operaciones:

- Se genera aleatoriamente un tensor

```
z
```

con dimensiones

```
[n_example, n_colors, width, height, 2]
```

a partir de una distribución normal estándar.

Esta operación utiliza una semilla de generación cuya longitud será análoga al término KL de un VAE.

- El tensor se procesa mediante una red neuronal denominada

```
equivariant_NN
```

que produce un tensor de logits con dimensiones

```
[n_example, n_colors, width, height, 2]
```

3. Finalmente se obtiene el rompecabezas muestreando colores de la distribución de probabilidad definida por dichos logits, produciendo un tensor de dimensiones

```
[n_example, width, height, 2]
```

Esperamos que los colores generados coincidan con los del rompecabezas real.

Este segundo proceso también utiliza una semilla, cuya longitud resulta análoga al término de reconstrucción de un VAE.

La red neuronal **equivariant_NN** posee una propiedad fundamental: es **equivariante** respecto a las transformaciones válidas de los rompecabezas ARC-AGI.

Esto significa que si:

- se permuta el orden de los ejemplos;
- se permutan los colores;
- se rota o refleja el tensor de entrada;

entonces la salida de la red sufrirá exactamente la misma transformación.

Algoritmo 1. Plantilla de un programa corto

La plantilla del programa realiza, para cada rompecabezas, las siguientes operaciones:

1. Definir la arquitectura de la red neuronal equivariante.
2. Insertar una semilla codificada (**seed_z**) correspondiente al rompecabezas.
3. Generar el tensor de entrada aleatorio **z**.
4. Insertar los pesos entrenados de la red neuronal.
5. Ejecutar una propagación hacia delante (*forward pass*).
6. Insertar una segunda semilla (**seed_error**).
7. Generar el rompecabezas completo.
8. Imprimir el rompecabezas.

Posteriormente el mismo bloque de código se repite para todos los rompecabezas del conjunto de datos.

La plantilla permite incorporar dos semillas pseudoaleatorias distintas para cada rompecabezas.

Manipulando adecuadamente la segunda semilla puede garantizarse que el rompecabezas generado coincida exactamente con el rompecabezas verdadero del conjunto de datos, independientemente de cuál haya sido la primera semilla utilizada.

Una vez garantizado esto, la longitud total del programa depende únicamente del número de bits necesarios para representar ambas semillas.

Por tanto, buscar programas cortos equivale simplemente a encontrar semillas lo más cortas posible.

Existen varias partes del programa que pueden modificarse para reducir esa longitud:

- las semillas de las líneas 3 y 7;
- los pesos de la red neuronal de la línea 5;
- la propia arquitectura definida en la línea 1.

Las dos primeras cuestiones se estudian en esta sección y la arquitectura se abordará en la Sección 4.

3.2 Optimización de las semillas

El Algoritmo 2 presenta un procedimiento para optimizar simultáneamente las semillas y los pesos del Algoritmo 1 con el objetivo de minimizar la longitud total del programa.

El procedimiento sigue tres pasos principales.

En primer lugar, modifica la semilla correspondiente a la generación del tensor **z** para que el muestreo aparente proceder de una distribución normal distinta, aprendida durante la optimización.

Posteriormente manipula la segunda semilla para garantizar que el rompecabezas generado coincida exactamente con el rompecabezas objetivo.

Finalmente aplica descenso por gradiente sobre:

- los parámetros de la distribución normal;

- los pesos de la red neuronal;

con el objetivo de minimizar la longitud total de las semillas.

La idea de manipular una semilla aleatoria para simular el muestreo desde otra distribución diferente presenta numerosas dificultades técnicas.

Bajo determinadas condiciones favorables esto puede realizarse mediante **Rejection Sampling**, que produce muestras de la distribución objetivo utilizando una semilla cuya longitud esperada depende del cociente máximo entre ambas distribuciones.

Posteriormente esta técnica puede mejorarse mediante **Relative Entropy Coding (REC)**, reduciendo la longitud esperada de la semilla hasta aproximarse a la divergencia de Kullback-Leibler (KL) entre ambas distribuciones.

No obstante, ejecutar REC requiere un tiempo exponencial respecto a dicha divergencia.

Para evitar este enorme coste computacional, CompressARC no ejecuta realmente REC.

En su lugar simula directamente las consecuencias que tendría dicho procedimiento.

Concretamente:

- el tensor z se genera directamente desde la distribución aprendida;
- la longitud de la primera semilla se aproxima mediante la divergencia KL entre la distribución aprendida y la distribución normal estándar;
- la longitud de la segunda semilla se aproxima mediante la entropía cruzada entre los logits producidos por la red y el rompecabezas conocido.

La demostración matemática de esta aproximación se presenta en el Apéndice B.

Algoritmo 2. Minimización de la Longitud de Descripción (MDL)

Entrada:

- Conjunto de datos ARC-AGI.

Procedimiento:

1. Definir la arquitectura de la red neuronal equivariante.
2. Para cada rompecabezas:
 - Inicializar aleatoriamente los pesos de la red.
 - Obtener las dimensiones del rompecabezas.
 - Inicializar una distribución normal parametrizada por media y covarianza diagonal.
3. Durante cada iteración:
 - Manipular la primera semilla para simular muestras procedentes de la distribución aprendida.
 - Generar el tensor latente z .
 - Ejecutar la red neuronal.
 - Manipular la segunda semilla para obtener exactamente el rompecabezas deseado.
 - Generar el rompecabezas completo.
 - Calcular la longitud total de ambas semillas.
 - Actualizar los parámetros mediante el optimizador Adam.
4. Insertar en el programa plantilla las semillas y los pesos obtenidos.
5. Repetir el procedimiento para todos los rompecabezas.
6. Devolver finalmente el programa comprimido.

Algoritmo 3. CompressARC

CompressARC es una versión simplificada del Algoritmo 2.

En lugar de realizar la manipulación explícita de semillas mediante REC, sustituye dicho procedimiento por una aproximación diferenciable.

El algoritmo funciona de la siguiente manera:

1. Inicializar la arquitectura.

2. Para cada rompecabezas:

- Inicializar los pesos de la red.
- Inicializar la distribución latente.

3. Repetir durante el entrenamiento:

- Muestrear directamente

```
z ~ N(μ, Σ)
```

- Ejecutar la red neuronal.
- Calcular la pérdida

```
KL(N(μ, Σ) || N(0, I))  
+  
CrossEntropy(logits, P)
```

que aproxima la longitud del programa.

- Actualizar los parámetros mediante Adam.

4. Una vez finalizado el entrenamiento:

- Muestrear la solución producida por la red.
- Añadir dicha solución al conjunto de rompecabezas resueltos.

5. Devolver todas las soluciones obtenidas.

Gracias a esta formulación, CompressARC puede comprimir automáticamente el conjunto de datos ARC-AGI refinando sucesivamente el programa plantilla hasta producir las soluciones.

El único componente que aún queda por definir es la arquitectura concreta de la red neuronal utilizada dentro de dicha plantilla, cuestión que constituye el contenido de la siguiente sección.

4. Arquitectura

La idea central detrás del diseño de la arquitectura de la red neuronal consiste en maximizar la probabilidad de generar los rompecabezas de ARC-AGI mediante muestreo. Al conseguirlo, disminuye la longitud de las semillas y, en consecuencia, la longitud total del programa descrito en el Algoritmo 1.

De acuerdo con la estructura de dicho algoritmo, esto implica que la red neuronal debe poseer **buenos sesgos inductivos (inductive biases)** para transformar ruido aleatorio en rompecabezas que parezcan plausibles dentro del dominio de ARC-AGI.

Es importante destacar que los rompecabezas del conjunto de entrenamiento **no intervienen directamente** en nuestro método. Su única utilidad consiste en ayudar al diseño manual de estos sesgos inductivos dentro de la arquitectura.

Equivarianza

Un rompecabezas de ARC-AGI sigue siendo esencialmente el mismo independientemente de:

- el orden en que aparezcan los ejemplos;
- el orden asignado a los colores;
- la orientación de la cuadrícula;
- las reflexiones horizontales o verticales.

Por tanto, deseamos que la red neuronal asigne inicialmente la **misma probabilidad** a todas estas variantes.

Para conseguirlo, diseñamos una arquitectura **equivariante** respecto a:

- permutaciones de los ejemplos;
- permutaciones de colores;
- rotaciones;
- reflexiones.

La equivarianza garantiza que, si la entrada experimenta cualquiera de estas transformaciones, la salida de la red sufrirá exactamente la misma transformación.

En consecuencia, cualquier asimetría presente en un rompecabezas concreto no está codificada en la arquitectura de la red, sino que debe introducirse modificando la semilla aleatoria utilizada para generar el tensor latente **z**, inclinando así la distribución hacia una solución concreta.

Estructura general de la red neuronal

La Figura 3 muestra la estructura principal de la arquitectura utilizada por CompressARC.

La red trabaja sobre una representación denominada **multitensor**, que será descrita con detalle posteriormente.

La arquitectura está formada por tres grandes bloques:

1. **Capa de decodificación**, que desempeña un papel similar al de una matriz de embeddings.
2. **Núcleo principal (core)**, basado en una conexión residual.
3. **Capa lineal de salida**, aplicada sobre la dimensión correspondiente a los canales.

Dentro del núcleo residual, distintas proyecciones lineales leen información desde el estado residual y la envían hacia operaciones especialmente diseñadas para ARC-AGI.

Una vez procesada, la información vuelve a incorporarse al estado residual mediante otra proyección lineal.

A lo largo de toda la red aparecen también operaciones de normalización.

El bloque completo del núcleo se repite **cuatro veces**.

Comparación con Transformers

La filosofía general recuerda a la arquitectura Transformer.

En ambos casos existe:

- un flujo residual donde se almacena la información;
- operaciones especializadas que leen y escriben sobre dicho flujo;
- una capa lineal final encargada de producir la salida.

La diferencia fundamental es que, en lugar de utilizar mecanismos de atención (*attention*) sobre secuencias de tokens, CompressARC incorpora operaciones específicamente diseñadas para manipular rompecabezas bidimensionales.

Operaciones especializadas

Las principales operaciones implementadas por la arquitectura son:

1. Suma y difusión (*broadcast*)

Permite sumar un tensor a lo largo de uno o varios ejes y posteriormente propagar el resultado a dimensiones superiores.

Esta operación facilita compartir información global entre distintas partes del rompecabezas.

2. Softmax

La función Softmax puede aplicarse sobre uno o varios ejes del multitensor.

Esto permite construir distribuciones de probabilidad adaptadas a diferentes dimensiones del problema.

3. Máximo acumulado (*Cumulative Max*)

Calcula máximos acumulados a lo largo de una dimensión geométrica.

Esta operación rompe deliberadamente determinadas simetrías geométricas, proporcionando capacidades espaciales específicas.

4. Desplazamiento (*Shift*)

Desplaza el contenido un píxel a lo largo de una dimensión espacial.

Este tipo de operación resulta especialmente útil para resolver numerosos rompecabezas de ARC-AGI donde aparecen traslaciones de objetos.

5. No linealidad elemento a elemento

Se aplica una función no lineal independiente sobre cada componente del tensor.

6. Normalización

Se realiza una normalización sobre la dimensión de canales.

Además de estas operaciones, la arquitectura incorpora otra operación adicional descrita en el Apéndice D.5.

Parámetros del modelo

Un aspecto interesante de la arquitectura es que las operaciones descritas anteriormente **no poseen parámetros propios**.

Su comportamiento está completamente determinado por las proyecciones lineales que leen y escriben sobre el estado residual.

Estas proyecciones trabajan únicamente sobre la dimensión de canales.

Para reducir el coste computacional:

- algunas partes del flujo residual utilizan **16 canales**;
- otras utilizan únicamente **8 canales**.

Como consecuencia, el número total de parámetros del modelo es de aproximadamente **76 000**, una cifra extraordinariamente pequeña para un sistema capaz de resolver problemas de ARC-AGI.

El formato Multitensor

Aunque anteriormente describimos los datos mediante un único tensor, la implementación real utiliza una estructura mucho más rica denominada **multitensor**.

En lugar de trabajar con un único tensor de dimensiones

```
[n_example,
 n_colors,
 width,
 height,
 channel]
```

la red mantiene simultáneamente numerosos tensores distintos.

Cada uno contiene únicamente un subconjunto de dichas dimensiones.

Por ejemplo, pueden existir tensores con forma:

```
[n_colors,
 width,
 channel]
```

o

```
[n_example,
 height,
 channel]
```

El tensor de entrada **z**, los logits producidos por la red y todas las activaciones internas adoptan este formato multitensor.

En términos generales, existe un tensor para prácticamente cada subconjunto posible de dimensiones.

Este diseño permite incorporar importantes sesgos inductivos.

Por ejemplo, una correspondencia entre columnas de la cuadrícula y determinados colores puede almacenarse naturalmente mediante un tensor con dimensiones:

```
[color,
 width,
 channel]
```

Los autores remiten al Apéndice C para una descripción completa del funcionamiento interno de los multitensores.

5. Resultados

Para cada rompecabezas se ejecutaron **2 000 iteraciones de entrenamiento durante la inferencia**, con un tiempo aproximado de **20 minutos por rompecabezas**.

Bajo este presupuesto computacional, CompressARC consiguió resolver correctamente:

- **20 %** de los rompecabezas del conjunto de evaluación.
- **34,75 %** de los rompecabezas del conjunto de entrenamiento.

La Figura 4 muestra cómo aumenta la precisión conforme se permite un mayor número de iteraciones de entrenamiento durante la inferencia.

Las Tablas 4 y 5 del Apéndice contienen los resultados numéricos completos, incluyendo los tiempos de ejecución.

Tabla 1. Comparación con otros métodos

Los autores comparan CompressARC con distintos enfoques existentes para resolver ARC-AGI.

Cada método dispone de dos intentos de solución por rompecabezas.

Una solución únicamente se considera correcta cuando:

- la forma de la cuadrícula coincide exactamente;
- todos los colores de todos los píxeles son correctos.

Como referencia adicional se incluye un modelo U-Net entrenado únicamente durante la inferencia utilizando los ejemplos del propio rompecabezas de prueba, con el fin de mantener las mismas condiciones que CompressARC.

Los resultados son los siguientes:

Método	Datos utilizados	Red neuronal	Precisión
Adivinanza aleatoria	Ninguno	No	0 %
Búsqueda exhaustiva de reglas	Ninguno	No	40 % (evaluación privada)
U-Net	Solo rompecabezas objetivo	Sí	0,75 %
CompressARC	Solo rompecabezas objetivo	Sí	20 %
HRM (ablación)	Rompecabezas de prueba	Sí	31 %
HRM	Entrenamiento + prueba	Sí	40,3 %
OpenAI o3 High	Datos a escala de Internet	Sí	87,5 %

5.1 ¿Qué rompecabezas puede y no puede resolver CompressARC?

CompressARC intenta explotar al máximo las capacidades que posee hasta encontrarse con alguna limitación de su arquitectura.

Los autores ilustran esta situación mediante el rompecabezas **28e73c20** del conjunto de entrenamiento.

En este problema es necesario prolongar un patrón desde el borde hacia el centro de la cuadrícula.

Las capas de la red permiten extender patrones correctamente únicamente a distancias relativamente cortas.

En consecuencia, CompressARC prolonga correctamente el patrón durante varios píxeles, pero cuando debe alcanzar el centro de la cuadrícula ya no dispone de suficiente capacidad para continuar de forma fiable y termina realizando una estimación.

El Apéndice H recoge de forma sistemática las capacidades que los autores han observado experimentalmente que CompressARC posee y aquellas que todavía no consigue realizar.

5.2 Estudio de caso: "Color the Boxes"

El siguiente rompecabezas analizado por los autores recibe el nombre de **Color the Boxes**.

En él, varias cajas deben colorearse dependiendo de la posición que ocupan dentro de la cuadrícula.

5.2 Estudio de caso: "Color the Boxes"

El rompecabezas mostrado en la Figura 5 recibe el nombre de **Color the Boxes**.

En este problema, cada caja debe colorearse según el lado de la cuadrícula en el que se encuentra.

Solución humana

Una persona resolvería este rompecabezas aproximadamente de la siguiente manera.

En primer lugar, observa que la cuadrícula de entrada está dividida en varias cajas y que esas mismas cajas siguen existiendo en la salida; la única diferencia es que ahora aparecen coloreadas.

El siguiente paso consiste en descubrir qué color corresponde a cada caja.

Al analizar los ejemplos proporcionados se observan varios patrones:

- Las esquinas siempre son negras.
- La caja central siempre es de color magenta.
- El color de las cajas situadas en los laterales depende de la dirección en la que se encuentran respecto al centro:
 - arriba → rojo;
 - abajo → azul;
 - derecha → verde;
 - izquierda → amarillo.

Una vez identificada esta regla, basta con copiar la cuadrícula de entrada y colorear cada caja según dichas correspondencias.

Solución obtenida por CompressARC

La Tabla 2 muestra cómo evoluciona el comportamiento de CompressARC durante el entrenamiento realizado en tiempo de inferencia.

Una vez finalizado el aprendizaje, los autores analizan la distribución latente **z** aprendida y descubren que esta codifica esencialmente dos tipos de información:

- una tabla que relaciona colores con direcciones;
- las posiciones de las líneas divisorias entre filas y columnas.

Durante el entrenamiento, el error de reconstrucción disminuye extremadamente rápido.

Aunque posteriormente permanece bajo durante la mayor parte del proceso, ocasionalmente aparecen picos de error.

Cuando esto ocurre, la divergencia KL correspondiente al espacio latente aumenta temporalmente, tal y como puede observarse en la Figura 6(a).

5.2.1 Análisis de la solución

Los autores estudian las representaciones almacenadas dentro del espacio latente **z** para comprender cómo aprende CompressARC a resolver el rompecabezas.

Recordemos que **z** no es un único tensor, sino un **multitensor**.

Cada tensor que forma parte del multitensor contribuye de manera independiente al valor total de la divergencia KL.

Analizando la contribución individual de cada tensor (Figura 6(b)) es posible identificar cuáles contienen realmente información útil para representar el rompecabezas.

Durante el entrenamiento ocurre algo especialmente interesante.

Prácticamente todos los tensores terminan reduciendo su contenido informativo hasta cero.

Sin embargo, **cuatro tensores** conservan información significativa.

En algunas repeticiones del experimento incluso uno de estos cuatro tensores llega a perder toda su información.

Cuando eso sucede, CompressARC normalmente ya no consigue recuperar la solución correcta.

La ejecución mostrada por los autores corresponde a un caso afortunado en el que uno de dichos tensores casi pierde toda su información, pero vuelve a recuperarla aproximadamente alrededor de la iteración 200.

Precisamente a partir de ese momento comienzan a aparecer muestras generadas con los colores correctos en las cajas adecuadas.

Qué representa cada tensor

Los autores inspeccionan la salida media de la capa de decodificación asociada a cada tensor del espacio latente.

Cada tensor contiene vectores de dimensión igual al número de canales.

Aplicando **Análisis de Componentes Principales (PCA)** sobre dichos vectores puede determinarse cuántos conceptos distintos codifica cada tensor.

Los resultados muestran lo siguiente.

Tensor [ejemplo, altura, canal] y [ejemplo, anchura, canal]

El componente principal dominante identifica automáticamente las posiciones de las filas y columnas que delimitan las cajas.

Concretamente:

- los píxeles con mayor intensidad indican la posición de las líneas horizontales;
- los píxeles con menor intensidad indican la posición de las líneas verticales.

De esta manera la red aprende la estructura geométrica del rompecabezas.

Tensor [dirección, color, canal]

El componente principal dominante de este tensor representa directamente la correspondencia entre direcciones y colores.

Los autores encuentran la siguiente asociación:

- azul ↔ arriba;
- verde ↔ izquierda;
- rojo ↔ abajo;
- amarillo ↔ derecha.

Este tensor indica qué color debe utilizarse para cada dirección correspondiente a las cajas situadas alrededor del centro.

La primera componente principal es **829 veces** más importante que la segunda, indicando que prácticamente toda la información relevante se concentra en una única dimensión.

Interpretación

Este análisis muestra que CompressARC no memoriza simplemente la solución final.

En su lugar aprende representaciones internas compactas que describen:

- la geometría del rompecabezas;
- la relación entre direcciones y colores;
- la posición de los separadores de filas y columnas.

Posteriormente reconstruye la solución utilizando dichas representaciones comprimidas.

6. Discusión

La dependencia del aprendizaje profundo moderno respecto a enormes cantidades de datos de alta calidad limita seriamente su aplicación a problemas donde los datos disponibles son escasos.

Este problema resulta especialmente evidente en ARC-AGI.

En este benchmark dominan actualmente los grandes modelos de lenguaje entrenados sobre conjuntos de datos especialmente ampliados, seleccionados y cuidadosamente preparados.

En este contexto presentamos CompressARC, un sistema que no solo prescinde completamente de los datos de entrenamiento, sino que elimina incluso la fase completa de preentrenamiento.

Intuitivamente cabría esperar que un enfoque así fuera incapaz de resolver absolutamente ningún rompecabezas.

Sin embargo, al aplicar el principio de Longitud Mínima de Descripción (MDL) directamente sobre el rompecabezas objetivo durante la inferencia, CompressARC consigue resolver una fracción sorprendentemente elevada de los problemas de ARC-AGI-1.

Fundamentos teóricos

La base teórica de CompressARC consiste en minimizar la longitud de una descripción algorítmica del rompecabezas objetivo.

Las estrategias tradicionales basadas en MDL apenas se han utilizado debido a que el espacio de búsqueda de programas posibles resulta prácticamente inabordable.

CompressARC evita este problema restringiendo la búsqueda a un espacio formado por redes neuronales optimizable mediante descenso por gradiente.

Aunque la arquitectura utilizada ha sido cuidadosamente diseñada a mano, su extraordinaria capacidad para generalizar utilizando únicamente dos pares entrada–salida sitúa a CompressARC en un régimen de generalización completamente distinto al observado habitualmente en ARC-AGI.

Trabajo futuro

Los autores consideran que la principal línea de mejora consiste en aumentar la eficiencia computacional.

La arquitectura utiliza numerosas operaciones personalizadas.

Implementaciones optimizadas mediante:

- compilación JIT;
- kernels CUDA fusionados;

permitirían acelerar significativamente cada iteración de entrenamiento.

Estas mejoras resultarían especialmente beneficiosas para rompecabezas de gran tamaño, ya que el tiempo de ejecución de la arquitectura aumenta con el número de píxeles presentes en la cuadrícula.

Conclusión

Los autores cuestionan la hipótesis ampliamente aceptada de que la inteligencia debe surgir necesariamente del preentrenamiento masivo y del uso de enormes volúmenes de datos.

En su lugar muestran que una utilización inteligente del principio de Longitud Mínima de Descripción y de las técnicas de compresión de información puede producir capacidades sorprendentemente elevadas.

CompressARC constituye una prueba de concepto que demuestra que los marcos modernos de aprendizaje profundo pueden combinarse con MDL para ofrecer una vía alternativa —y complementaria— hacia el desarrollo de una Inteligencia Artificial General (AGI).

Apéndice A. Trabajo relacionado

En esta sección revisamos los trabajos más relevantes relacionados con CompressARC.

Nuestro método se sitúa en la intersección de varias áreas de investigación:

- ARC-AGI;
- aprendizaje en tiempo de inferencia (*test-time learning*);
- principio de Longitud Mínima de Descripción (MDL);
- codificación mediante entropía relativa (Relative Entropy Coding, REC);
- aprendizaje profundo con escasez extrema de datos.

A.1 ARC-AGI

Desde la publicación del benchmark ARC-AGI por Chollet (2019), se han propuesto numerosos enfoques para resolver este conjunto de problemas.

En términos generales, estos métodos pueden agruparse en tres grandes categorías.

Sistemas simbólicos

Los primeros trabajos se centraron principalmente en sistemas simbólicos.

Estos métodos intentan descubrir explícitamente una secuencia de transformaciones capaz de convertir las cuadrículas de entrada en las cuadrículas de salida.

Entre las técnicas utilizadas se encuentran:

- motores de búsqueda sobre programas;
- gramáticas de transformaciones;
- sistemas basados en reglas;
- lenguajes específicos de dominio (DSL).

Una ventaja importante de estos métodos consiste en que pueden generalizar con muy pocos ejemplos.

Sin embargo, el espacio de búsqueda suele crecer muy rápidamente, haciendo difícil resolver rompecabezas complejos.

Métodos basados en aprendizaje profundo

Otra línea de investigación consiste en entrenar redes neuronales utilizando el conjunto de entrenamiento de ARC-AGI.

En este caso la red intenta aprender regularidades comunes presentes en numerosos rompecabezas diferentes.

Diversos trabajos han empleado arquitecturas como:

- CNN;
- Transformers;
- Graph Neural Networks;
- modelos híbridos.

Estos enfoques suelen obtener mejores resultados cuando existe suficiente cantidad de datos de entrenamiento.

Sin embargo, su capacidad de generalización disminuye considerablemente cuando aparecen reglas completamente nuevas.

Grandes modelos de lenguaje

Los avances recientes en grandes modelos de lenguaje han permitido obtener resultados muy superiores.

Estos sistemas utilizan:

- enormes cantidades de datos de preentrenamiento;
- aprendizaje mediante instrucciones;
- cadenas de razonamiento (*Chain of Thought*);
- búsqueda durante la inferencia.

Aunque alcanzan niveles de precisión muy elevados, resulta difícil determinar hasta qué punto dichas capacidades proceden realmente del razonamiento o del conocimiento adquirido durante el preentrenamiento.

CompressARC explora precisamente el extremo opuesto.

Nuestro sistema elimina completamente el preentrenamiento y utiliza únicamente el propio rompecabezas que está resolviendo.

A.2 Aprendizaje durante la inferencia

El aprendizaje durante la inferencia (*test-time learning*) ha recibido una atención creciente durante los últimos años.

En estos métodos el modelo continúa adaptando sus parámetros después de haber finalizado el entrenamiento convencional.

Las aplicaciones incluyen:

- adaptación a cambios de distribución (*distribution shift*);
- personalización;
- metaaprendizaje;
- aprendizaje continuo.

CompressARC lleva esta idea mucho más lejos.

En nuestro caso:

- no existe entrenamiento previo;
- todos los parámetros comienzan con valores aleatorios;
- el aprendizaje completo ocurre exclusivamente durante la inferencia.

Hasta donde sabemos, no existen trabajos anteriores que entrenen una red neuronal desde cero utilizando únicamente un único rompecabezas de ARC-AGI y aun así consigan una capacidad apreciable de generalización.

A.3 Longitud Mínima de Descripción (MDL)

El principio de Longitud Mínima de Descripción fue introducido por Rissanen (1978).

La idea fundamental consiste en que la mejor explicación para un conjunto de datos es aquella que permite describirlos utilizando el menor número posible de bits.

Desde entonces MDL ha sido ampliamente utilizado en áreas como:

- selección de modelos;
- inferencia estadística;
- compresión de datos;
- aprendizaje automático.

CompressARC aplica este principio de una forma diferente.

En lugar de seleccionar entre varios modelos previamente entrenados, optimiza directamente un programa capaz de reconstruir el rompecabezas completo.

El objetivo de entrenamiento coincide exactamente con la longitud de dicho programa.

A.4 Codificación mediante Entropía Relativa (REC)

Relative Entropy Coding (REC) es una técnica desarrollada para generar muestras procedentes de una distribución objetivo utilizando una fuente de aleatoriedad diferente.

La longitud esperada de la información adicional necesaria depende de la divergencia de Kullback-Leibler entre ambas distribuciones.

REC proporciona una interpretación especialmente elegante del término KL presente en los Autoencoders Variacionales.

Sin embargo, su coste computacional aumenta exponencialmente con la divergencia.

Por este motivo CompressARC no ejecuta literalmente REC.

En su lugar optimiza directamente una función de pérdida equivalente que aproxima el número esperado de bits requerido por dicho procedimiento.

Una demostración formal de esta equivalencia aparece en el Apéndice B.

A.5 Aprendizaje con cantidades extremadamente reducidas de datos

La mayoría de los métodos modernos de aprendizaje profundo requieren grandes cantidades de ejemplos etiquetados.

Se han propuesto numerosas estrategias para reducir esta dependencia.

Entre ellas destacan:

- aprendizaje auto-supervisado;
- metaaprendizaje;
- aprendizaje por transferencia;
- aumento de datos (*data augmentation*);
- modelos fundacionales preentrenados.

CompressARC adopta una estrategia completamente distinta.

No intenta reutilizar conocimientos obtenidos previamente.

En su lugar utiliza el principio de compresión para inducir directamente la estructura presente en el rompecabezas objetivo.

Esta filosofía resulta especialmente adecuada para dominios donde apenas existen datos disponibles y donde las reglas cambian continuamente entre instancias.

Resumen del trabajo relacionado

En conjunto, los trabajos anteriores muestran dos tendencias claramente diferenciadas.

Por un lado, los sistemas simbólicos ofrecen una excelente eficiencia en cuanto al uso de datos, pero presentan dificultades para escalar.

Por otro, los modelos neuronales modernos escalan con gran facilidad, aunque normalmente necesitan enormes conjuntos de entrenamiento.

CompressARC intenta combinar las ventajas de ambos enfoques.

Utiliza una arquitectura neuronal optimizable mediante descenso por gradiente, pero guiando el aprendizaje mediante el principio clásico de Longitud Mínima de Descripción en lugar de depender del preentrenamiento masivo.

Apéndice B. Derivación de la función objetivo

En este apéndice demostramos que la función de pérdida utilizada por CompressARC constituye una aproximación diferenciable de la longitud del programa descrito en el Algoritmo 1.

La demostración se basa en los resultados conocidos sobre **Relative Entropy Coding (REC)**.

B.1 Longitud esperada de la primera semilla

Recordemos que el tensor latente

$$\mathbf{z}$$

se genera originalmente a partir de una distribución normal estándar

$$\mathcal{N}(\mathbf{0}, \mathbf{I}) .$$

Sin embargo, durante la optimización deseamos generar muestras procedentes de otra distribución

$$q(z) \text{ .}$$

Una posibilidad consiste en utilizar **Relative Entropy Coding**.

REC permite transformar una secuencia de bits aleatorios obtenidos de la distribución base en muestras distribuidas según la distribución objetivo.

El resultado fundamental establece que la longitud esperada de la información adicional necesaria es aproximadamente

$$\text{KL}(q(z) \parallel N(0, I)) \text{ .}$$

Por tanto, el número esperado de bits requerido para codificar la primera semilla coincide con la divergencia KL entre ambas distribuciones.

Este término aparece exactamente igual que en un Autoencoder Variacional.

CompressARC utiliza directamente esta cantidad como aproximación diferenciable de la longitud de dicha semilla.

B.2 Longitud esperada de la segunda semilla

Una vez generado el tensor latente

$$z_r$$

la red neuronal produce una distribución categórica sobre los posibles colores de cada píxel.

Denotemos por

$$p(x|z)$$

la probabilidad asignada al rompecabezas correcto.

Si utilizáramos nuevamente REC para forzar que el resultado generado coincidiera exactamente con el rompecabezas objetivo, la longitud esperada de la segunda semilla sería

$$-\log p(x|z) \text{ .}$$

Cuando la distribución es categórica, esta cantidad coincide exactamente con la pérdida de entropía cruzada (*cross-entropy*).

En consecuencia, el término de reconstrucción utilizado por CompressARC constituye una aproximación directa del número esperado de bits necesarios para codificar dicha semilla.

B.3 Longitud total del programa

Puesto que ambas semillas son independientes, la longitud total esperada del programa puede aproximarse simplemente sumando ambas contribuciones:

$$\begin{aligned} \text{Longitud} &\approx \\ &\text{KL}(q(z) \parallel N(0, I)) \\ &+ \\ &\text{CrossEntropy}(x, \text{logits}) \text{ .} \end{aligned}$$

Esta expresión coincide exactamente con la función de pérdida optimizada durante el entrenamiento.

De este modo, minimizar la pérdida equivale a minimizar una aproximación diferenciable de la longitud del programa.

B.4 Relación con los Autoencoders Variacionales

La similitud entre CompressARC y un Autoencoder Variacional resulta evidente.

Ambos optimizan una función objetivo formada por dos términos:

- una divergencia KL;
- un término de reconstrucción.

Sin embargo, la interpretación es completamente distinta.

En un VAE convencional:

- la divergencia KL actúa como regularizador;
- el término de reconstrucción mide el error de reconstrucción de los datos.

En CompressARC ambos términos representan directamente cantidades de información:

- la longitud de la primera semilla;
- la longitud de la segunda semilla.

Por tanto, la función objetivo no se introduce como una heurística de regularización, sino como una estimación explícita de la longitud del programa buscado.

B.5 Aproximación diferenciable

La ejecución literal del Algoritmo 2 exigiría implementar Relative Entropy Coding durante cada iteración de entrenamiento.

Este procedimiento tendría un coste computacional prohibitivo.

En lugar de ello, CompressARC sustituye REC por una aproximación diferenciable.

Durante el entrenamiento:

1. se generan directamente muestras de la distribución aprendida;
2. se añade el término KL como estimación del coste de modificar la primera semilla;
3. se utiliza la entropía cruzada como estimación del coste de modificar la segunda semilla.

Esta sustitución permite aplicar descenso por gradiente convencional.

Los experimentos muestran que dicha aproximación resulta suficientemente precisa para optimizar la longitud del programa de forma efectiva.

B.6 Interpretación desde la teoría de la información

Desde un punto de vista de teoría de la información, CompressARC intenta responder a la siguiente pregunta:

¿Cuál es el menor número posible de bits necesarios para describir este rompecabezas concreto?

La respuesta se obtiene repartiendo la información entre tres componentes:

- la arquitectura de la red neuronal;
- los pesos aprendidos;
- las dos semillas pseudoaleatorias.

La arquitectura permanece fija.

Los pesos se optimizan para capturar regularidades compartidas dentro del propio rompecabezas.

Las semillas únicamente almacenan la información que no puede describirse mediante dichas regularidades.

Como consecuencia, cuanto mejor consiga la red explicar la estructura del rompecabezas, menor será la información residual que deberán almacenar las semillas.

Esto constituye exactamente el principio de Longitud Mínima de Descripción.

B.7 Consecuencias prácticas

Esta formulación presenta varias propiedades interesantes.

En primer lugar, la función objetivo posee una interpretación probabilística rigurosa.

No se trata simplemente de una pérdida diseñada empíricamente, sino de una aproximación explícita al número esperado de bits necesarios para representar el programa.

En segundo lugar, el entrenamiento deja de depender de conjuntos de datos externos.

Toda la información utilizada procede exclusivamente del rompecabezas que se desea resolver.

Finalmente, el enfoque conecta tres áreas que normalmente se estudian por separado:

- compresión de información;
- Autoencoders Variacionales;
- aprendizaje durante la inferencia.

CompressARC muestra que estas tres perspectivas pueden integrarse dentro de un único marco matemático para resolver problemas de razonamiento abstracto.

Apéndice C. Multitensores

En este apéndice describimos con detalle la estructura de datos denominada **multitensor**, utilizada internamente por CompressARC.

Los multitensores constituyen el núcleo de la arquitectura propuesta y permiten representar relaciones entre diferentes subconjuntos de las dimensiones presentes en un rompecabezas de ARC-AGI.

C.1 Motivación

Una representación convencional almacena toda la información en un único tensor de alta dimensionalidad.

Por ejemplo, podríamos utilizar un tensor con dimensiones:

```
[ejemplo,
color,
anchura,
altura,
canal]
```

Sin embargo, esta representación presenta varios inconvenientes.

Muchas relaciones importantes únicamente dependen de un pequeño subconjunto de dichas dimensiones.

Por ejemplo:

- la relación entre un color y una columna;
- la posición de una línea horizontal;
- la correspondencia entre distintos ejemplos;
- la distribución de colores.

Obligar a que toda esta información se almacene dentro de un único tensor dificulta el aprendizaje y obliga a la red neuronal a redescubrir continuamente dichas relaciones.

Para evitar este problema, CompressARC utiliza una colección de tensores independientes.

Cada tensor representa únicamente un subconjunto concreto de dimensiones.

A esta colección de tensores la denominamos **multitensor**.

C.2 Definición formal

Sea

```
D = {
ejemplo,
color,
anchura,
altura
}
```

el conjunto de dimensiones disponibles.

Para cada subconjunto

$S \subseteq D$

se define un tensor independiente

T_S

cuyas dimensiones corresponden exactamente a los elementos de dicho subconjunto, junto con una dimensión adicional correspondiente a los canales.

Por ejemplo:

$T_\emptyset$

representa un único vector global.

Mientras que

$T_{\{\text{color}\}}$

contiene un vector para cada color.

Del mismo modo,

$T_{\{\text{anchura}, \text{ altura}\}}$

almacena un vector independiente para cada posición espacial de la cuadrícula.

En conjunto, todos estos tensores forman el multitensor.

C.3 Número de tensores

Dado que existen cuatro dimensiones principales,

$$2^4 = 16$$

subconjuntos diferentes.

Por tanto, el multitensor contiene dieciséis tensores distintos.

Estos tensores representan todos los niveles posibles de abstracción.

Por ejemplo:

- información completamente global;
- información asociada únicamente a un ejemplo;
- únicamente a un color;
- únicamente a una posición;
- combinaciones entre varias dimensiones.

C.4 Inicialización

El tensor latente

z

se genera directamente en formato multitensor.

Cada uno de los dieciséis tensores se inicializa mediante una distribución normal independiente.

Durante el entrenamiento, cada tensor aprende únicamente la información necesaria para describir el rompecabezas.

Esto produce una representación extremadamente eficiente.

Muchos tensores terminan convergiendo hacia distribuciones prácticamente idénticas a la normal estándar y, por tanto, apenas contribuyen a la divergencia KL.

En consecuencia, únicamente unos pocos tensores almacenan realmente información relevante.

C.5 Compartición de información

Las operaciones utilizadas por CompressARC permiten transferir información entre distintos tensores del multitenzor.

Por ejemplo, una operación puede:

- leer información desde un tensor definido sobre colores;
- combinarla con otro tensor definido sobre posiciones;
- escribir posteriormente el resultado sobre el estado residual.

Esta forma de comunicación permite construir representaciones muy ricas sin necesidad de aumentar significativamente el número de parámetros.

C.6 Ventajas

Los autores destacan varias ventajas importantes del formato multitenzor.

Representaciones más compactas

Cada concepto puede almacenarse únicamente sobre las dimensiones donde realmente resulta necesario.

Esto reduce considerablemente la redundancia.

Mejor sesgo inductivo

El diseño obliga implícitamente a la red neuronal a separar conceptos diferentes.

Por ejemplo:

- información espacial;
- información cromática;
- información correspondiente a distintos ejemplos.

Esta separación facilita el aprendizaje.

Mayor interpretabilidad

Resulta relativamente sencillo inspeccionar cada tensor por separado y determinar qué tipo de información contiene.

Precisamente esta propiedad permitió realizar el análisis del espacio latente presentado en la Sección 5.

Escalabilidad

Aunque el número de tensores aumenta exponencialmente con el número de dimensiones disponibles,

cada tensor individual permanece relativamente pequeño.

En la práctica, el coste computacional continúa siendo manejable.

C.7 Relación con otras arquitecturas

Los multitensores presentan ciertas similitudes con otras representaciones utilizadas anteriormente.

Por ejemplo:

- tensores factorizados;
- representaciones jerárquicas;
- grafos heterogéneos;
- arquitecturas con múltiples escalas.

No obstante, CompressARC emplea una construcción mucho más sistemática.

Cada subconjunto posible de dimensiones posee su propia representación explícita.

Las operaciones de la red permiten mover información libremente entre dichos subconjuntos.

C.8 Papel dentro de CompressARC

El objetivo principal del multitensor consiste en facilitar la compresión.

Cuando una determinada regularidad únicamente depende de unas pocas dimensiones, puede almacenarse directamente en el tensor correspondiente.

Por ejemplo:

- una correspondencia entre colores y direcciones no necesita información espacial completa;
- la posición de una línea horizontal no depende del color;
- una propiedad global del rompecabezas puede almacenarse en el tensor vacío.

Gracias a esta organización, la información necesaria para describir el rompecabezas queda distribuida entre distintos niveles de abstracción.

Ello reduce la cantidad de información residual que debe codificarse mediante las semillas y, por tanto, disminuye la longitud total del programa.

C.9 Resumen

El multitensor constituye uno de los elementos fundamentales de CompressARC.

En lugar de utilizar una única representación monolítica, la arquitectura mantiene simultáneamente una familia completa de representaciones especializadas.

Cada una captura regularidades asociadas únicamente a un subconjunto concreto de dimensiones.

Esta organización proporciona:

- mejores sesgos inductivos;
- mayor interpretabilidad;
- una representación mucho más eficiente desde el punto de vista de la compresión de información.

Los siguientes apéndices describen las operaciones concretas que actúan sobre estos multitensores y la implementación detallada de la arquitectura neuronal.

Apéndice D. Detalles de la arquitectura

En este apéndice describimos con precisión todas las operaciones que forman la arquitectura neuronal de CompressARC.

Aunque la estructura general de la red es relativamente sencilla, gran parte de su capacidad proviene del diseño específico de estas operaciones y de la manera en que intercambian información entre los distintos tensores del multitensor.

D.1 Flujo residual

Al igual que en los Transformers modernos, CompressARC mantiene un **estado residual** que atraviesa toda la red.

Cada bloque realiza siempre el mismo patrón de operaciones:

1. Leer información del estado residual.
2. Transformarla mediante una operación especializada.
3. Escribir el resultado de nuevo sobre el estado residual.
4. Sumar la actualización mediante una conexión residual.

Gracias a esta estructura, diferentes operaciones pueden cooperar sin destruir la información previamente aprendida.

D.2 Proyecciones lineales

Cada operación especializada está rodeada por dos transformaciones lineales:

- una proyección de entrada;
- una proyección de salida.

La primera adapta la representación residual al formato requerido por la operación.

La segunda vuelve a proyectar el resultado hacia el espacio residual común.

Es importante destacar que las propias operaciones especializadas prácticamente no contienen parámetros entrenables.

La mayor parte de los parámetros del modelo se concentra precisamente en estas proyecciones lineales.

D.3 Operación Broadcast (Difusión)

La operación **Broadcast** permite compartir información entre tensores definidos sobre diferentes subconjuntos de dimensiones.

Su funcionamiento puede resumirse en dos pasos.

Paso 1. Reducción

Primero se agregan valores eliminando una o varias dimensiones mediante operaciones de suma.

Por ejemplo, un tensor definido sobre:

```
[color, anchura]
```

puede reducirse a uno definido únicamente sobre:

```
[color]
```

sumando todos los valores correspondientes a las distintas columnas.

Paso 2. Difusión

Posteriormente el resultado se replica sobre nuevas dimensiones hasta alcanzar el formato requerido.

Por ejemplo:

```
[color]
```

puede expandirse nuevamente hasta

```
[color, anchura]
```

copiando el mismo vector en todas las columnas.

Esta operación permite construir fácilmente información global o compartir características entre distintas partes del rompecabezas.

D.4 Softmax

CompressARC utiliza funciones Softmax sobre distintas dimensiones del multitenso

Dependiendo del contexto, Softmax puede aplicarse:

- sobre colores;
- sobre posiciones espaciales;
- sobre ejemplos;
- sobre cualquier otra dimensión disponible.

Esto permite generar distribuciones de probabilidad adaptadas a diferentes tipos de razonamiento.

Por ejemplo, una Softmax aplicada sobre los colores puede representar directamente la probabilidad de cada color para un píxel determinado.

D.5 Máximo acumulado (Cumulative Max)

La operación **Cumulative Max** calcula máximos acumulados a lo largo de una dimensión.

Formalmente, cada posición contiene el máximo observado hasta ese punto.

Por ejemplo, dada la secuencia:

```
1 4 2 3 5
```

el máximo acumulado produce:

```
1 4 4 4 5
```

Este operador introduce una direccionalidad que resulta muy útil para numerosos rompecabezas donde intervienen:

- crecimiento de objetos;
- propagación;
- expansión;
- detección de bordes.

Además, rompe deliberadamente algunas simetrías espaciales que de otro modo resultarían difíciles de aprender.

D.6 Desplazamiento (Shift)

La operación **Shift** desplaza un tensor exactamente un píxel en una dirección determinada.

Puede aplicarse sobre cualquiera de las dimensiones espaciales.

Los desplazamientos posibles incluyen:

- arriba;
- abajo;
- izquierda;
- derecha.

Los valores que quedan fuera de la cuadrícula se rellenan con ceros.

Esta operación proporciona una forma muy eficiente de modelar transformaciones espaciales frecuentes en ARC-AGI.

Muchos rompecabezas consisten precisamente en trasladar objetos manteniendo inalterada su forma.

D.7 Funciones no lineales

Después de determinadas operaciones se aplica una función de activación elemento a elemento.

Estas funciones permiten introducir relaciones no lineales entre las distintas características aprendidas.

Su comportamiento es completamente independiente para cada componente del tensor.

D.8 Normalización

CompressARC utiliza capas de normalización aplicadas sobre la dimensión correspondiente a los canales.

El objetivo consiste en mantener una escala numérica estable durante el entrenamiento.

Ello facilita la optimización mediante descenso por gradiente y evita que determinadas operaciones dominen al resto.

D.9 Bloque residual completo

Cada bloque residual sigue la siguiente secuencia general:

1. Normalización.
2. Proyección lineal.
3. Operación especializada.
4. Proyección lineal de salida.
5. Suma residual.

Este patrón se repite para todas las operaciones presentes dentro del núcleo de la arquitectura.

Como resultado, el estado residual va acumulando progresivamente representaciones cada vez más útiles para reconstruir el rompecabezas.

D.10 Repetición de bloques

El núcleo completo se repite cuatro veces.

Los autores observaron experimentalmente que un número reducido de bloques resulta suficiente para capturar las regularidades presentes en ARC-AGI.

Incrementar significativamente la profundidad apenas mejoraba la precisión, mientras que aumentaba considerablemente el tiempo de entrenamiento durante la inferencia.

Por este motivo se optó por una arquitectura relativamente poco profunda pero cuidadosamente diseñada.

Apéndice D (continuación). Detalles de la arquitectura

D.11 Composición de operaciones

Una característica importante de la arquitectura es que ninguna operación especializada pretende resolver por sí sola un rompecabezas completo.

Cada operación únicamente introduce una capacidad elemental.

Por ejemplo:

- desplazar información;
- propagar información;
- seleccionar máximos;
- combinar dimensiones;
- normalizar representaciones.

La inteligencia emerge de la composición repetida de estas operaciones dentro del flujo residual.

Esta filosofía recuerda al diseño de los Transformers, donde la atención y las redes feed-forward son relativamente simples de manera individual, pero generan un comportamiento complejo al combinarse.

D.12 Escritura sobre el estado residual

Después de ejecutar una operación especializada, el resultado nunca sustituye completamente al estado residual.

En su lugar, se proyecta nuevamente sobre el espacio residual y se suma a la representación existente.

Formalmente:

```
Residual ← Residual + Projection( Operation( Projection( Residual ) ) )
```

Este mecanismo permite que distintas operaciones colaboren sin destruir la información producida por las anteriores.

Como consecuencia, el entrenamiento resulta considerablemente más estable.

D.13 Dimensión de canales

La dimensión correspondiente a los canales constituye el único espacio donde realmente se realizan transformaciones aprendidas.

Todas las proyecciones lineales actúan exclusivamente sobre esta dimensión.

Las dimensiones estructurales del multitenzor (ejemplo, color, anchura y altura) permanecen intactas durante dichas transformaciones.

Esta separación entre:

- estructura;
- representación;

simplifica notablemente la arquitectura.

D.14 Distribución de parámetros

La mayor parte de los aproximadamente **76 000 parámetros** del modelo se concentra en:

- proyecciones lineales de entrada;
- proyecciones lineales de salida.

En cambio:

- Broadcast;
- Shift;
- Cumulative Max;
- Softmax;

- Normalización;

aportan muy pocos o ningún parámetro entrenable.

Esto implica que la capacidad del modelo depende mucho más del diseño de sus operaciones que del número de parámetros.

D.15 Eficiencia computacional

Muchas de las operaciones utilizadas recorren únicamente una dimensión del multitensores.

En consecuencia, su coste computacional crece aproximadamente de forma lineal con el tamaño de dicha dimensión.

Esto permite entrenar la red durante la inferencia incluso para rompecabezas relativamente grandes.

No obstante, el tiempo total continúa aumentando con el número de píxeles presentes en la cuadrícula.

Por ello los autores consideran que futuras implementaciones optimizadas podrían ofrecer mejoras significativas de velocidad.

D.16 Implementación vectorizada

Todas las operaciones fueron implementadas utilizando tensores completamente vectorizados.

No se realizan bucles explícitos sobre los píxeles del rompecabezas.

Este enfoque permite aprovechar eficientemente las operaciones masivas disponibles en aceleradores como GPU.

Además, simplifica la diferenciación automática utilizada por el descenso por gradiente.

D.17 Inicialización

Todos los pesos de la red neuronal se inicializan aleatoriamente al comienzo de cada rompecabezas.

No existe ningún conocimiento compartido entre rompecabezas diferentes.

Esto significa que, al comenzar la optimización de un nuevo rompecabezas, la red se encuentra exactamente en el mismo estado que tendría un modelo completamente nuevo.

Toda la información adquirida durante el entrenamiento anterior se descarta.

D.18 Optimización

Los parámetros se optimizan utilizando el algoritmo **Adam**.

Durante cada iteración se actualizan simultáneamente:

- los parámetros de la distribución latente;
- los pesos de todas las proyecciones lineales.

La arquitectura permanece fija durante todo el proceso.

D.19 Decisiones de diseño

Los autores destacan varias decisiones adoptadas tras numerosos experimentos preliminares.

Entre ellas:

- mantener una arquitectura relativamente pequeña;
- favorecer operaciones con significado geométrico;
- evitar introducir demasiados parámetros;
- preservar las simetrías presentes en ARC-AGI siempre que fuera posible.

Según los autores, estas decisiones permiten obtener una excelente relación entre capacidad de generalización y longitud del programa.

D.20 Interpretación

Desde el punto de vista del principio de Longitud Mínima de Descripción, la arquitectura constituye una parte fija del programa.

Cuanto mejor sea dicha arquitectura describiendo regularidades habituales de ARC-AGI, menos información adicional deberán almacenar:

- los pesos entrenados;
- la distribución latente;
- las semillas pseudoaleatorias.

Por ello, el diseño manual de buenos sesgos inductivos resulta equivalente a reducir el tamaño fijo del programa.

Resumen del Apéndice D

El diseño de CompressARC sigue una filosofía claramente distinta a la de los grandes modelos de lenguaje.

En lugar de incrementar continuamente el número de parámetros, los autores dedican el esfuerzo al diseño de operaciones cuidadosamente elegidas para manipular directamente las estructuras geométricas presentes en ARC-AGI.

El resultado es una arquitectura muy pequeña, altamente interpretable y especialmente adecuada para ser optimizada mediante el principio de Longitud Mínima de Descripción.

Las operaciones descritas en este apéndice constituyen la base sobre la que posteriormente se apoyan todos los experimentos presentados en el artículo.

Apéndice E. Detalles experimentales

En este apéndice se describen los detalles de implementación utilizados para todos los experimentos presentados en el artículo.

El objetivo es facilitar la reproducibilidad de los resultados obtenidos por CompressARC.

E.1 Inicialización

Cada rompecabezas comienza exactamente desde el mismo estado inicial.

Concretamente:

- todos los pesos de la red neuronal se inicializan aleatoriamente;
- la distribución latente comienza próxima a una distribución normal estándar;
- no se reutiliza ningún parámetro aprendido anteriormente.

Esto garantiza que cada rompecabezas constituye un proceso de aprendizaje completamente independiente.

E.2 Optimizador

Todos los experimentos utilizan el optimizador **Adam**.

Durante cada iteración se optimizan simultáneamente:

- los pesos de la red;
- los parámetros de la distribución latente.

La arquitectura permanece completamente fija.

No se modifica ninguna operación del modelo durante el entrenamiento.

E.3 Número de iteraciones

Cada rompecabezas se optimiza durante aproximadamente:

2000 iteraciones

Este número representa un equilibrio entre:

- precisión;
- coste computacional.

Los autores observaron que aumentar considerablemente el número de iteraciones produce mejoras adicionales, aunque cada vez menores.

E.4 Tiempo de ejecución

El tiempo empleado depende principalmente de:

- el tamaño de la cuadrícula;
- el número de ejemplos disponibles;
- la complejidad del rompecabezas.

En promedio, cada rompecabezas requiere alrededor de:

20 minutos

en el hardware utilizado por los autores.

E.5 Criterio de parada

No se emplea un criterio dinámico de parada temprana (*early stopping*).

Todos los rompecabezas reciben exactamente el mismo presupuesto de optimización.

Esta decisión simplifica la comparación entre experimentos.

E.6 Muestreo de soluciones

Una vez finalizado el entrenamiento durante la inferencia, CompressARC genera las soluciones directamente a partir de la distribución aprendida.

Cada intento corresponde a una muestra obtenida del modelo.

Como ocurre en la evaluación oficial de ARC-AGI, se permiten dos intentos por rompecabezas.

E.7 Reproducibilidad

Todos los experimentos utilizan:

- la misma arquitectura;
- el mismo procedimiento de entrenamiento;
- los mismos hiperparámetros.

No se ajustan parámetros específicos para rompecabezas individuales.

Apéndice F. Estudios de ablación

Con el fin de comprender qué componentes resultan realmente importantes, los autores realizan diversos estudios de ablación.

En cada experimento eliminan o modifican uno de los componentes principales de CompressARC y vuelven a medir la precisión obtenida.

F.1 Eliminación de la equivarianza

La primera ablación consiste en eliminar las propiedades de equivarianza de la arquitectura.

En este caso la red deja de tratar de manera equivalente:

- diferentes permutaciones de colores;
- rotaciones;
- reflexiones;
- cambios en el orden de los ejemplos.

Los resultados muestran una disminución apreciable del rendimiento.

Esto confirma que incorporar explícitamente las simetrías presentes en ARC-AGI constituye uno de los principales sesgos inductivos del modelo.

F.2 Eliminación del multitensor

En esta variante el multitensor se sustituye por una representación convencional basada en un único tensor.

La precisión disminuye de forma significativa.

Los autores concluyen que distribuir la información entre distintos subconjuntos de dimensiones facilita considerablemente la compresión del rompecabezas.

F.3 Eliminación del término KL

Otra ablación consiste en eliminar completamente el término correspondiente a la divergencia KL.

En este caso la red únicamente intenta reconstruir perfectamente los ejemplos conocidos.

El resultado es un fuerte sobreajuste (*overfitting*).

Aunque la reconstrucción de los ejemplos de entrenamiento resulta excelente, la calidad de las soluciones generadas para los ejemplos de prueba disminuye notablemente.

Este experimento pone de manifiesto que el término KL actúa realmente como una presión para encontrar descripciones más simples.

F.4 Sustitución de las operaciones especializadas

Los autores sustituyen varias de las operaciones diseñadas específicamente para ARC-AGI por transformaciones lineales convencionales.

La precisión vuelve a reducirse.

Ello indica que operaciones como:

- Shift;
- Broadcast;
- Cumulative Max;

aportan capacidades difíciles de reproducir mediante capas densas estándar.

F.5 Incremento del número de parámetros

También se evalúan modelos considerablemente más grandes.

Sorprendentemente, aumentar el número de parámetros no produce mejoras proporcionales.

En algunos casos incluso empeora la capacidad de generalización.

Según los autores, esto ocurre porque modelos excesivamente grandes pueden memorizar con demasiada facilidad los ejemplos conocidos sin descubrir representaciones realmente compactas.

F.6 Conclusión de las ablaciones

Los estudios realizados muestran que prácticamente todos los componentes principales de CompressARC contribuyen al rendimiento final.

En particular destacan tres elementos fundamentales:

1. la formulación basada en Longitud Mínima de Descripción;
2. la arquitectura equivariante;
3. la representación mediante multitensores.

La combinación de estos tres componentes resulta considerablemente más eficaz que cualquiera de ellos por separado.

Apéndice G. Análisis cualitativo de los resultados

Además de las métricas cuantitativas presentadas en el artículo principal, los autores realizaron un análisis detallado de los rompecabezas resueltos y no resueltos con el fin de identificar patrones comunes.

En lugar de limitarse a medir la precisión global, estudiaron qué tipos de transformaciones aprende realmente CompressARC.

Este análisis permite comprender mejor tanto las fortalezas como las limitaciones actuales del método.

G.1 Rompecabezas resueltos correctamente

CompressARC obtiene buenos resultados en problemas donde la solución puede describirse mediante regularidades relativamente compactas.

Entre los ejemplos más habituales se encuentran:

- propagación de colores;
- detección de regiones;
- prolongación de patrones cortos;
- copias y desplazamientos de objetos;
- simetrías simples;
- identificación de componentes conectados;
- transformaciones geométricas locales.

En muchos de estos casos la representación aprendida resulta fácilmente interpretable mediante el análisis del espacio latente.

G.2 Errores frecuentes

Los fallos observados presentan varios patrones comunes.

En algunos rompecabezas la red identifica correctamente la regla general, pero comete pequeños errores en posiciones concretas.

En otros casos produce una solución casi correcta, aunque uno o varios objetos aparecen desplazados algunos píxeles.

También existen ejemplos donde la estructura global resulta correcta, pero determinados colores son intercambiados.

Estos errores sugieren que la arquitectura ha aprendido una representación próxima a la solución correcta, aunque todavía insuficientemente precisa.

G.3 Evolución durante el entrenamiento

Uno de los aspectos más interesantes observados por los autores es que la calidad de las soluciones no mejora siempre de forma monotónica.

Durante el entrenamiento aparecen fases donde:

- el error disminuye rápidamente;
- posteriormente aumenta;
- vuelve a reducirse más adelante.

Este comportamiento coincide con las variaciones observadas en la divergencia KL.

Los autores interpretan estos cambios como reorganizaciones internas de la representación latente.

Apéndice H. Capacidades y limitaciones

Este apéndice resume qué habilidades parecen emerger de la arquitectura propuesta y cuáles continúan representando un desafío.

Los autores insisten en que esta clasificación procede de la observación empírica de numerosos rompecabezas y no constituye una lista exhaustiva.

H.1 Capacidades observadas

CompressARC demuestra ser capaz de aprender tareas como:

- detectar objetos;
- identificar componentes conectados;
- propagar patrones;
- copiar regiones;
- trasladar objetos;
- completar figuras parcialmente visibles;
- aplicar simetrías sencillas;
- asociar colores con posiciones;
- utilizar información procedente de varios ejemplos simultáneamente;
- aprender representaciones geométricas compactas.

En muchos casos estas habilidades emergen únicamente durante la optimización realizada en tiempo de inferencia.

H.2 Limitaciones observadas

Por el contrario, los autores identifican varias limitaciones importantes.

Entre ellas destacan:

- dificultades para realizar razonamientos que requieren muchas etapas consecutivas;
- propagaciones a grandes distancias;
- dependencias globales muy complejas;
- conteos exactos de numerosos objetos;
- algoritmos iterativos largos;
- reglas que requieren memoria de largo alcance.

Estos problemas aparecen principalmente cuando la solución exige mantener información durante muchas transformaciones sucesivas.

H.3 Influencia de la arquitectura

Muchas de las limitaciones anteriores no parecen deberse al principio de Longitud Mínima de Descripción.

Según los autores, la causa principal reside en la capacidad limitada de la arquitectura neuronal utilizada.

Por ejemplo, una operación Shift únicamente desplaza la información un píxel por capa.

Con cuatro bloques residuales, la distancia máxima sobre la que puede propagarse información resulta relativamente pequeña.

Este hecho explica por qué algunos patrones dejan de extenderse correctamente cuando deben recorrer grandes regiones de la cuadrícula.

H.4 Posibles mejoras

Los autores proponen varias líneas para superar estas limitaciones.

Entre ellas:

- arquitecturas más profundas;
- nuevas operaciones especializadas;
- implementaciones más eficientes que permitan ejecutar muchas más iteraciones;
- mecanismos de propagación de largo alcance;
- mejores sesgos inductivos.

Esperan que estas mejoras aumenten considerablemente la capacidad del enfoque sin abandonar el marco basado en MDL.

Apéndice I. Análisis de las representaciones aprendidas

Una de las ventajas más importantes de CompressARC consiste en que sus representaciones internas resultan relativamente fáciles de interpretar.

A diferencia de muchos modelos de gran tamaño, los distintos componentes del multitensores pueden inspeccionarse individualmente.

I.1 Distribución de la información

Los autores observan que la mayor parte de los tensores terminan almacenando muy poca información.

Únicamente unos pocos contribuyen significativamente a la divergencia KL.

Esto indica que la representación aprendida es altamente dispersa (*sparse*).

La mayor parte de la estructura del rompecabezas queda concentrada en unos pocos tensores especializados.

I.2 Especialización espontánea

Durante el entrenamiento distintos tensores adquieren funciones claramente diferenciadas.

Por ejemplo:

- algunos representan relaciones espaciales;
- otros almacenan asociaciones entre colores;
- otros describen posiciones características;
- otros contienen información prácticamente global.

Esta especialización aparece espontáneamente como consecuencia de la optimización de la longitud del programa.

No se introduce explícitamente mediante ninguna restricción adicional.

I.3 Interpretabilidad

La posibilidad de inspeccionar cada tensor por separado facilita enormemente comprender el comportamiento del modelo.

Los autores consideran que esta interpretabilidad constituye una ventaja importante frente a arquitecturas mucho mayores donde la información aparece distribuida de forma mucho más difusa.

Además, permite diagnosticar con relativa facilidad por qué determinados rompecabezas no llegan a resolverse correctamente.

Apéndice J. Ejemplos adicionales

En este apéndice se presentan numerosos ejemplos adicionales de rompecabezas pertenecientes a ARC-AGI, junto con las soluciones producidas por CompressARC.

El objetivo es ilustrar visualmente el comportamiento del sistema en una amplia variedad de situaciones.

Cada ejemplo muestra:

- los pares de entrenamiento;
- la entrada de prueba;
- la solución generada por CompressARC;
- la solución correcta.

Los ejemplos incluyen tanto casos resueltos correctamente como ejemplos en los que el modelo comete errores.

Esta comparación permite apreciar con mayor claridad qué tipos de razonamiento consigue realizar actualmente la arquitectura.

J.1 Soluciones correctas

En los ejemplos correctamente resueltos se observa que CompressARC suele descubrir representaciones compactas capaces de explicar simultáneamente todos los ejemplos disponibles.

En muchos casos la solución aparece de forma relativamente temprana durante la optimización y posteriormente se mantiene estable durante el resto del entrenamiento.

J.2 Soluciones parcialmente correctas

También aparecen numerosos casos donde la solución es casi correcta.

Entre los errores más frecuentes destacan:

- un único píxel incorrecto;
- colores intercambiados;
- pequeños desplazamientos;
- errores localizados en regiones alejadas del resto de la estructura.

Estos ejemplos muestran que la representación aprendida suele capturar correctamente la regla principal, aunque determinados detalles permanecen sin resolver.

J.3 Casos no resueltos

Los rompecabezas completamente no resueltos suelen compartir alguna de las siguientes características:

- reglas muy largas;
- transformaciones altamente iterativas;
- dependencias globales complejas;
- necesidad de contar con precisión numerosos objetos;

- relaciones espaciales de largo alcance.

Estos casos coinciden con las limitaciones descritas previamente en el Apéndice H.

Apéndice K. Tablas completas de resultados

Este apéndice recopila los resultados numéricos completos obtenidos durante la evaluación.

Para cada rompecabezas se incluyen, entre otros datos:

- identificador del rompecabezas;
- número de iteraciones realizadas;
- tiempo de ejecución;
- longitud aproximada del programa;
- resultado obtenido;
- estado (correcto o incorrecto).

Las tablas completas permiten reproducir exactamente las estadísticas resumidas en el cuerpo principal del artículo.

K.1 Resultados sobre el conjunto de entrenamiento

La primera tabla contiene todos los rompecabezas pertenecientes al conjunto oficial de entrenamiento de ARC-AGI.

Puede observarse una variabilidad considerable entre rompecabezas.

Algunos convergen rápidamente hacia una solución correcta.

Otros requieren prácticamente todas las iteraciones disponibles.

También existen rompecabezas donde el entrenamiento no consigue encontrar una representación suficientemente compacta.

K.2 Resultados sobre el conjunto de evaluación

La segunda tabla presenta los resultados correspondientes al conjunto oficial de evaluación.

La precisión global obtenida coincide con los valores resumidos anteriormente.

Los autores incluyen igualmente los tiempos de ejecución individuales para facilitar futuras comparaciones con otros métodos.

K.3 Observaciones

Las tablas muestran que el coste computacional depende principalmente del tamaño del rompecabezas y no únicamente de su dificultad conceptual.

Algunos problemas conceptualmente sencillos requieren tiempos relativamente elevados simplemente porque contienen cuadrículas de mayor tamaño.

Apéndice L. Conclusiones finales

CompressARC demuestra que es posible abordar problemas de razonamiento abstracto utilizando un paradigma completamente diferente al dominante en los modelos actuales de gran escala.

En lugar de depender de enormes cantidades de datos de entrenamiento y de modelos con miles de millones de parámetros, el sistema:

- comienza desde parámetros aleatorios;
- aprende exclusivamente durante la inferencia;
- optimiza directamente una aproximación diferenciable de la longitud del programa que describe el rompecabezas.

Los experimentos muestran que este enfoque permite resolver una proporción significativa de los rompecabezas de ARC-AGI sin utilizar ningún conocimiento adquirido previamente.

Perspectiva

Los autores consideran que CompressARC representa únicamente un primer paso.

Existen numerosas posibilidades para mejorar el sistema mediante:

- arquitecturas más expresivas;
- operaciones especializadas adicionales;
- implementaciones más eficientes;
- mejores estrategias de optimización.

Todas estas mejoras podrían incorporarse manteniendo intacta la idea fundamental del trabajo:

Resolver un problema equivale a encontrar la descripción computacional más corta capaz de generarlo.

Mensaje principal del artículo

La principal aportación de CompressARC no consiste únicamente en el porcentaje de precisión alcanzado.

Su contribución más importante es demostrar experimentalmente que el principio de **Longitud Mínima de Descripción (Minimum Description Length, MDL)** puede utilizarse como objetivo de entrenamiento directo para redes neuronales entrenadas íntegramente durante la inferencia.

Este resultado abre una nueva línea de investigación situada entre:

- la teoría algorítmica de la información;
- la compresión de datos;
- el aprendizaje profundo;
- el razonamiento abstracto.

Los autores sugieren que esta combinación podría constituir una alternativa prometedora —o al menos complementaria— al paradigma basado exclusivamente en el escalado masivo de modelos y datos.